

Security Recommendations

Web Application Security Review & Vulnerability Analysis

Prepared By: Lakshmi Subhaja Kari

Assessment Year: 2026

Target: DVWA on Metasploitable 2

Target IP: 192.168.23.129

1. Introduction

This document provides remediation recommendations based on the security observations identified during the authorized Web Application Security Review.

The recommendations focus on reducing the application's attack surface, improving server security, protecting administrative resources, strengthening authentication and session controls, and implementing secure development practices.

2. High-Priority Recommendations

2.1 Upgrade Outdated Apache and PHP

Finding: Apache 2.2.8 and PHP 5.2.4 were identified during Nikto enumeration.

Risk: High

Recommendation:

- Upgrade Apache to a currently supported version.
- Upgrade PHP to a currently supported version.
- Apply security patches regularly.
- Remove unsupported software from production systems.
- Maintain an inventory of installed software versions.

Priority: P1 – High

2.2 Restrict Exposed phpMyAdmin

Finding: phpMyAdmin was identified through Nikto and Gobuster enumeration.

Risk: High

Recommendation:

- Remove phpMyAdmin if it is not required.
- Restrict access to authorized administrator networks.
- Use strong authentication.
- Apply network access controls.
- Keep phpMyAdmin updated.
- Monitor administrative access attempts.

Priority: P1 – High

3. Medium-Priority Recommendations

3.1 Disable Directory Listing

Finding: Directory indexing was identified on resources such as `/icons/`, `/doc/`, and `/test/`.

Risk: Medium

Recommendation:

Disable directory listing unless it is specifically required.

Unnecessary directories and files should also be removed from the server.

Priority: P2 – Medium

3.2 Remove phpinfo.php

Finding: `phpinfo.php` was accessible.

Risk: Medium

Recommendation:

Remove `phpinfo.php` and other diagnostic files from production environments.

Diagnostic information should only be available to authorized administrators when required.

Priority: P2 – Medium

3.3 Disable HTTP TRACE

Finding: HTTP TRACE was identified as enabled.

Risk: Medium

Recommendation:

Disable HTTP TRACE unless there is a documented operational requirement.

The web server configuration should be reviewed after making the change.

Priority: P2 – Medium

3.4 Implement Security Headers

Finding: Several recommended HTTP security headers were not detected.

Risk: Medium

Recommendation:

Implement appropriate security headers based on the application's requirements.

Recommended headers include:

- Content-Security-Policy
- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy
- Strict-Transport-Security when HTTPS is correctly deployed

Headers should be tested after implementation to confirm that they are correctly returned by the server.

Priority: P2 – Medium

4. Authentication Recommendations

The assessment included controlled authentication testing and did not confirm a specific authentication vulnerability.

Nevertheless, production applications should implement:

- Strong password requirements
- Login rate limiting
- Protection against repeated failed login attempts
- Secure authentication mechanisms
- Appropriate account protection
- Security monitoring and logging

Priority: P3 – Continuous Improvement

5. Session Management Recommendations

Session management was reviewed using Burp Suite.

Production applications should:

- Use Secure cookies when HTTPS is used.
- Use HttpOnly cookies to reduce client-side access to session identifiers.
- Configure SameSite appropriately.
- Regenerate session identifiers after authentication where appropriate.
- Expire inactive sessions.
- Invalidate sessions during logout.
- Avoid exposing session identifiers in URLs.

Priority: P3 – Continuous Improvement

6. XSS Prevention

Reflected and stored XSS were tested but were not confirmed during this assessment.

Applications should nevertheless implement:

- Context-aware output encoding
- Input validation
- Secure templating
- Content Security Policy
- Safe handling of user-generated content

Developers should avoid inserting untrusted input directly into HTML, JavaScript, CSS or other executable contexts.

Priority: P3 – Preventive Control

7. SQL Injection Prevention

SQL Injection was tested but was not confirmed during the performed test.

Recommended controls include:

- Parameterized queries
- Prepared statements
- Input validation
- Least-privilege database accounts
- Secure database error handling
- Regular database security reviews

Applications should never construct SQL queries directly from untrusted user input.

Priority: P3 – Preventive Control

8. File Upload Security

The file upload assessment was not completed.

For applications that support file uploads, recommended controls include:

- Allow only required file types.
- Validate file content rather than relying only on extensions.
- Rename uploaded files.
- Store uploaded files outside executable web directories where possible.
- Restrict file permissions.
- Apply file-size limits.
- Scan uploads where appropriate.
- Prevent uploaded files from being executed as server-side code.

A dedicated authorized test should be performed before considering the upload functionality fully assessed.

9. Remove Unnecessary Resources

The enumeration identified resources including test and documentation-related directories.

Production systems should remove or restrict:

- Test directories
- Development files
- Diagnostic pages
- Documentation directories
- Backup files
- Unused scripts
- Administrative interfaces

Reducing unnecessary resources reduces the application's attack surface.

10. Logging and Monitoring

Security-relevant events should be logged and monitored.

Recommended events include:

- Failed login attempts
- Successful administrative logins
- Access to administrative interfaces
- Unusual HTTP requests
- Repeated enumeration activity

- Server errors
- Configuration changes

Logs should be protected against unauthorized modification.

11. Patch Management

A regular patch-management process should be established.

The process should include:

1. Identify installed software.
2. Monitor security updates.
3. Test updates where necessary.
4. Apply security patches.
5. Verify successful deployment.
6. Document changes.
7. Repeat regularly.

12. Security Testing

Security testing should be performed:

- Before production deployment
- After major application changes
- After significant infrastructure changes
- After security patches
- Periodically as part of a security program

Testing should include vulnerability scanning, manual security review and validation of remediation.

13. Remediation Priority

Priority	Recommendation
P1	Upgrade outdated Apache/PHP
P1	Restrict/remove phpMyAdmin
P2	Disable directory listing

P2	Remove phpinfo.php
P2	Disable HTTP TRACE
P2	Implement security headers
P3	Strengthen authentication
P3	Harden session management
P3	Apply XSS prevention controls
P3	Apply SQL Injection prevention controls
P3	Remove unnecessary resources
P3	Improve logging and monitoring
P3	Perform regular security testing

14. Verification Checklist

After remediation, verify:

- Apache is running a supported version.
- PHP is running a supported version.
- phpMyAdmin is removed or restricted.
- Directory listing is disabled.
- phpinfo.php has been removed.
- HTTP TRACE is disabled.
- Security headers are configured.
- Authentication rate limiting is enabled.
- Session cookies have appropriate security attributes.
- Unnecessary test resources have been removed.
- Security logs are monitored.
- A follow-up security assessment has been completed.

15. Conclusion

The recommendations in this document are intended to reduce the security risks identified during the authorized web application security assessment.

The highest priorities are upgrading outdated server components and restricting exposed administrative functionality. Medium-priority improvements include disabling directory listing, removing information-disclosure resources, disabling unnecessary HTTP functionality and implementing security headers.

After remediation, a follow-up security assessment should be performed to confirm that the identified weaknesses have been successfully addressed.

Prepared By: Lakshmi Subhaja Kari

Project: Web Application Security Review & Vulnerability Analysis

Year: 2026